

21st Century Alchemy

Part 1: The birth of deep learning



Nicholas J. Cooper



Table of Contents

- Overview of this course
- What is an intelligent machine?
- The Perceptron
- Limits of perceptrons and the 1st quest for depth
- Backpropagation and the 1st architecture explosion

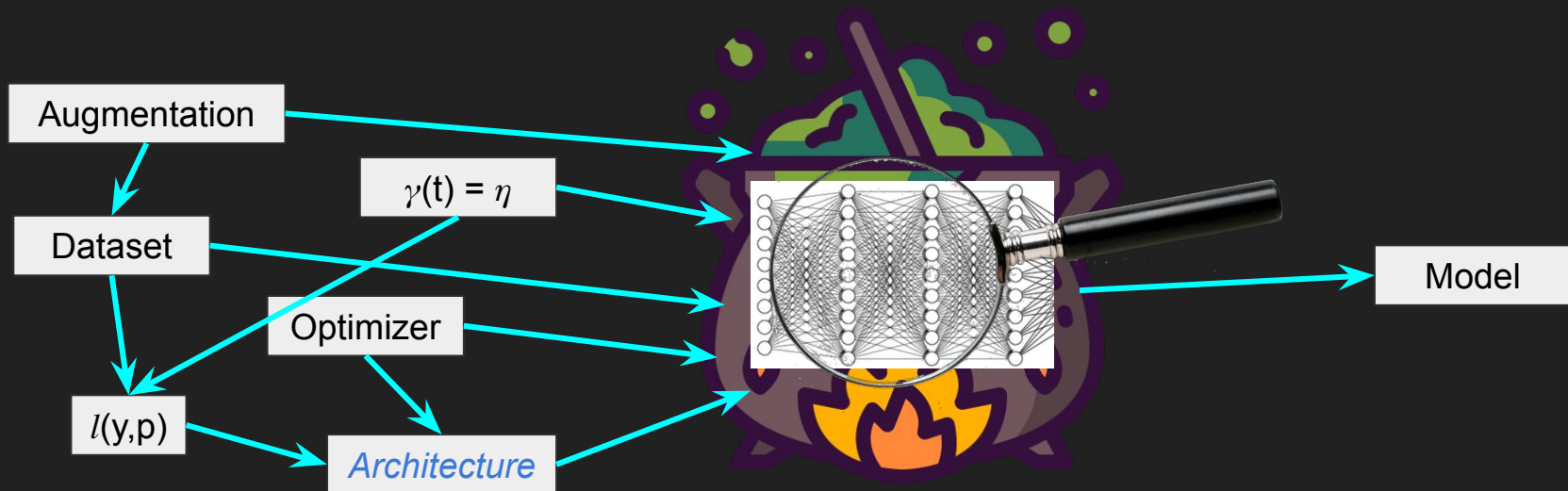
About Me

- Undergrad @ Rutgers University (B.S. in Electrical Engineering, B.S. in Math)
- Full time software engineer for ~3 years @ C360 (startup acquired by Cosm)
- Got bored...back to (grad) school!
 - University of Colorado $\Rightarrow$ startup (in progress)
 - Worked for grad. level course “foundations of neural networks and deep learning” since start

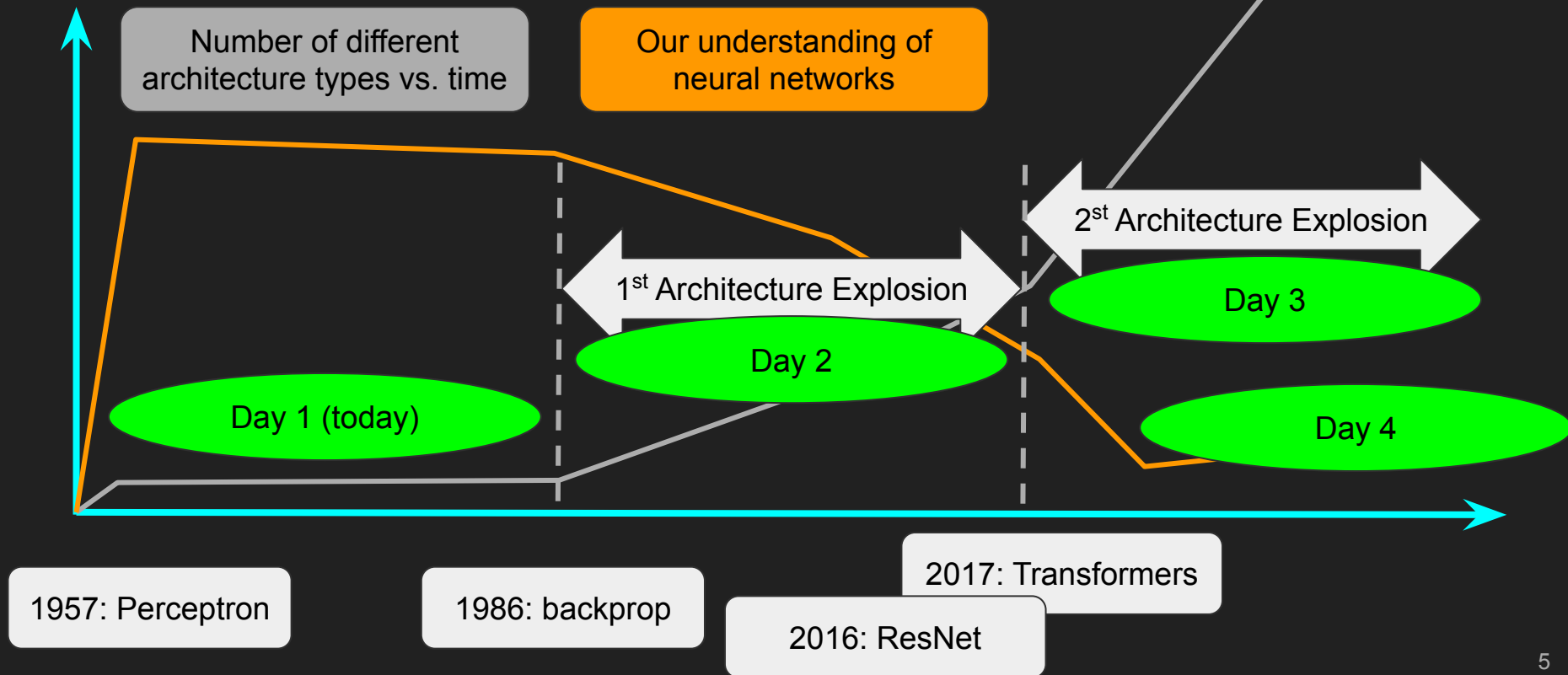


Overview of Course

- We will study deep neural networks from an *architectural perspective*



Overview of Course



Overview of Course

- Day 1: fun history lesson; math warm up, origins of deep neural networks
 - Neural networks were not “deep” until 1986
- Day 2: things get more involved; convolutional networks, vanishing gradients
 - Pure generalization achieved $\Rightarrow$ 1st architecture explosion
- Day 3: things get very involved; recurrent networks, attention, transformers
 - A new type of “layer” $\Rightarrow$ 2nd architecture explosion
- Day 4: a whirlwind tour of present day (2026) theories of deep neural nets
 - Generalization is hard to work with, formally

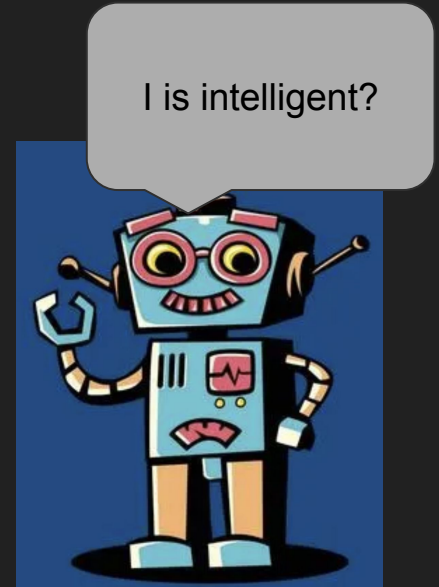
Intelligent Machines

Rewind the clock back to ~1950...



What is an intelligent machine?

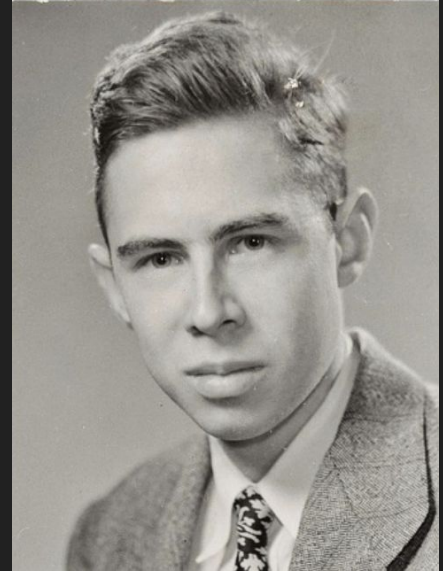
- A new perspective on computing was emerging
- Alan Turing's now famous *Turing Test* is a brand new idea
 - Can a machine convince a human that it is itself human?
 - Using only text-based communication
- While this test may seem simplistic today, at the time it represented a new way of thinking about computers
- Before, computers were seen simply as machines for performing routine, deterministic, algorithmic tasks



The Transition from Rule-Based to Data-Based Machines

- In 1957, Frank Rosenblatt published a technical report on “Project Para”
 - Work performed at Cornell Aeronautical Laboratory in Buffalo, NY
- A central aim of this project was to produce an intelligent system
- **Critically, intelligence was coupled with pattern recognition:**

“A primary requirement of such a [intelligent] system is that it must be able to recognize complex patterns...”



Rosenblatt in 1950

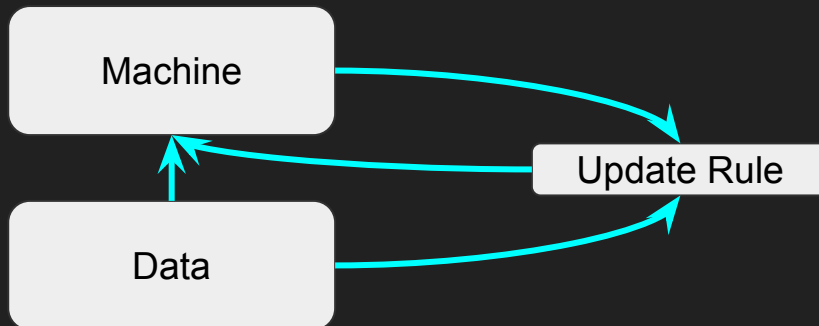
Quote from: Rosenblatt et. al., “The Perceptron: A Perceiving and Recognizing Automaton”, 1957
Picture credit: [Wikipedia](#)

Intelligence as a Statement about Data

- Prior to project para, the status quo was to “intelligent-ify” machines by explicitly programming rules. This is a *symbolic perspective* on intelligence
- The shift in focus to pattern recognition marks a significant moment in the history of deep learning
 - Rules are, fundamentally, properties of the *machine*
 - Patterns are, fundamentally, properties of the *data*
- We continue to witness the importance of this shift in perspective today

Data-Based Learning is Born

- In follow up work in 1958, Rosenblatt constructed and trained the first artificial neural network — the Perceptron.
- Regardless of the Perceptron's function as an *architecture*, its *training* marks a foundational moment in DL
- First instance of a machine learning patterns from data *without being explicitly programmed* — still the de-facto definition of machine learning used today



Why do we want Intelligent Machines, anyway?

- The late 1950s was near the peak of the Cold War era in the United States
 - Particularly relevant as Project Para was funded by the US Navy
- As a result, there was high interest in creating machines to process information faster than humans
- Immense external pressure had a great impact on the research conducted
- Moreover, the 1950s was a period of heightened interest in *cybernetics*
 - An interdisciplinary field of study that factorized into machine learning, robotics, control theory, and several other areas



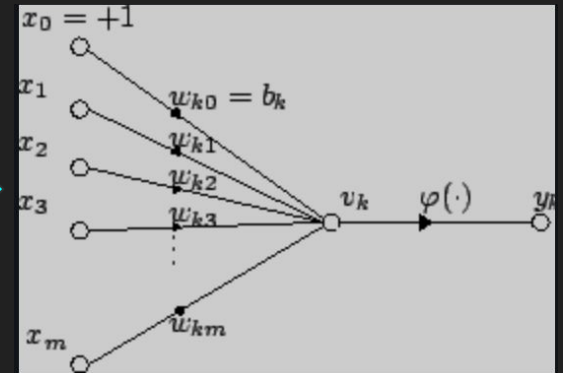
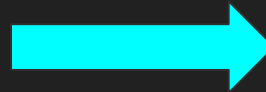
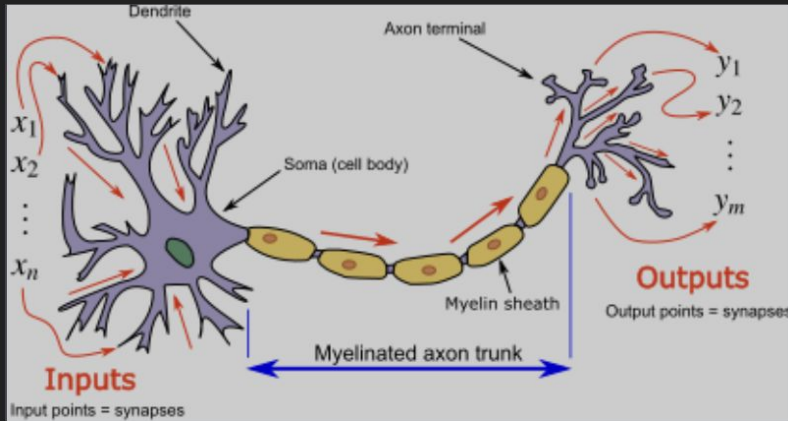
One of the first ever illustrations of a “cyborg” from **1569**

Picture credit: [Wikipedia](#)

The Perceptron Architecture

Artificial Neurons

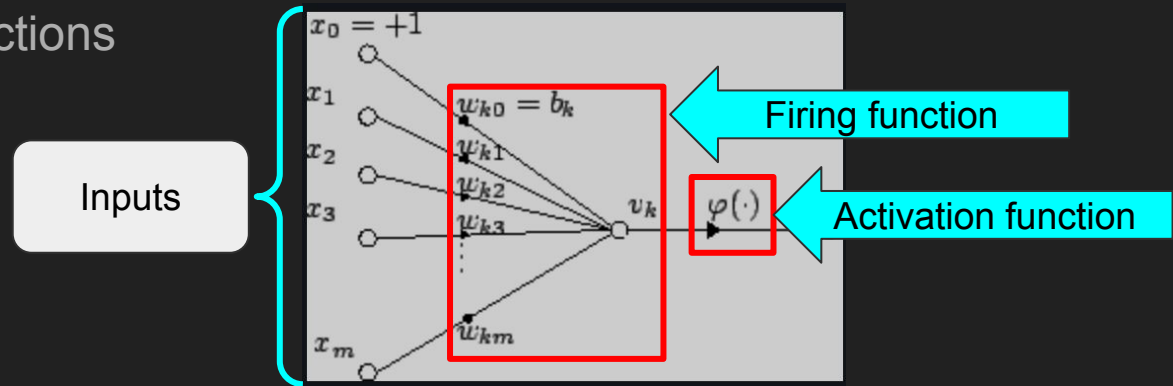
- Idea: build intelligent machines by computationalizing examples from biological systems
- The McCulloch-Pitts neuron is a computational model of a biological neuron
 - Introduced in the 1943 paper: “*A Logical Calculus of the Ideas Immanent in Nervous Activity*”



Picture credit: [Wikipedia](#)

Components of Neuron

- To construct a computational model of a neuron, one requires:
 - Inputs — data the neuron should respond to
 - Firing function — rule for deciding which inputs induce a response from the neuron
 - Activation function — decides when the neuron activates and sends an output signal
- For now, the inputs are treated as givens, we focus our attention to the firing and activation functions



Picture credit: [Wikipedia](#)

The Perceptron's Neuron Architecture

- The Perceptron used simple firing and activation functions:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

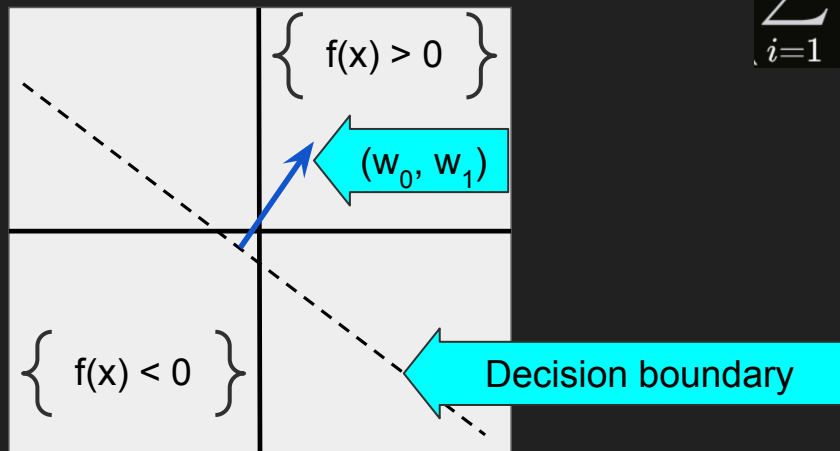
- The **weights** w_i control how the neuron responds to the inputs x_i
- The **bias** b sets a threshold for the “agreement” between the weights and inputs
- The **activation function** f decides if the neuron activates to send an output

Equations taken from: Michael Brenndoerfer, [“The Perceptron - Foundation of Modern Neural Networks”](#)

Geometry of the Perceptron

- A perceptron's activation is determined by the sides of a line (in general, a hyperplane)
- The values of the weights define the normal vector of the hyperplane
 - Firing function is positive/negative based on side of hyperplane
 - Firing function is zero at the *decision boundary*

$$\sum_{i=1}^n w_i x_i + b$$



What can a Perceptron do?

- Lines are good at dividing space into two parts
- Use perceptrons to separate out two types of inputs
- Can we use a perceptron to an input is red or blue?
 - What about if a letter is equal to “A” or not?
 - What about if a letter is equal to “B” or not?
 - ...
- Stack perceptrons in parallel to partition input space into more than 2 parts
 - Increase dimensionality of the system’s prediction/output space

Training the Perceptron

- We aim to teach Perceptrons to recognize complex patterns
- Idea:
 - Present a pattern to the machine as an input/output pair (X, Y)
 - X is input stimulus to the artificial neuron
 - Y is the known/ground truth type of the input
 - Evaluate the machine's output when presented with the pattern as input
 - If the output is not “correct”, modify the state of the machine
- Geometrically:
 - Move the position of the hyperplane to partition input space based on the positions of two types of points

How to train your Perceptron

- Send input to perceptron
- Compare predicted type to ground truth type
- if (prediction correct): do nothing
- if (prediction not correct): update weights and biases to make the prediction “less wrong” using an update rule
 - Increase/decrease similarity with each input
 - Amount of **parameter adjustment** is controlled by the **learning rate**
- Rinse and repeat

$$w_i^{new} = w_i^{old} + \alpha \cdot (target - output) \cdot x_i$$

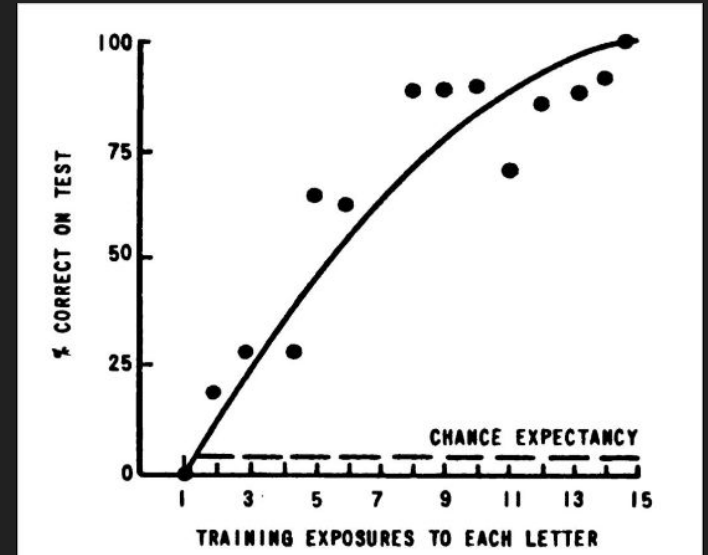
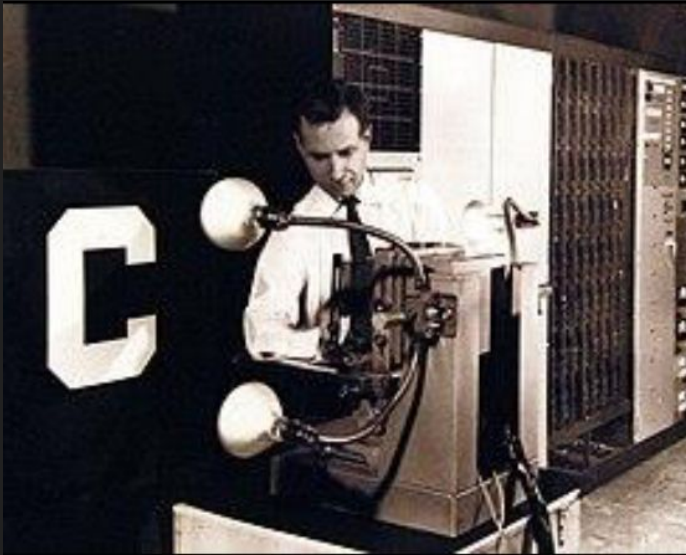
Equation taken from: Michael Brenndoerfer, [“The Perceptron - Foundation of Modern Neural Networks”](#)

How to train your Perceptron: Visualized

- Demo (code: https://github.com/combinatoriallabs/SEMF_26_Demos/)

It's Alive!

- Perceptron was able to “see” letters



Picture credits: [Wikipedia](#), Rosenblatt (1962)

Response to the Perceptron

- Rosenblatt's construction gathered much attention

The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence . . . Later Perceptrons will be able to recognize people and call out their names and instantly translate speech in one language to speech and writing in another language, it was predicted. (*New York Times*, 1958a, p. 25:2)

- This was one of many hype cycles surrounding artificial intelligence
 - Sound familiar?
- Today's LLM frenzy is just the latest hype cycle

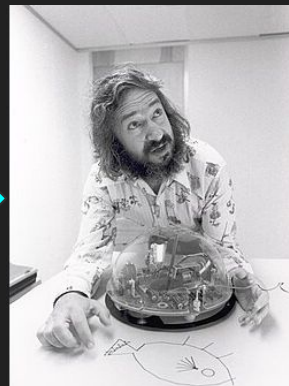
Limits of the Perceptron

Critics of Perceptron

- Broadly, there was pushback against Perceptron from symbolic researchers
 - Specifically, Marvin Minsky and Seymour Papert were unconvinced by the Perceptron's ability to recognize patterns
- In 1969 they published examples of patterns unlearnable by Perceptron



Minsky

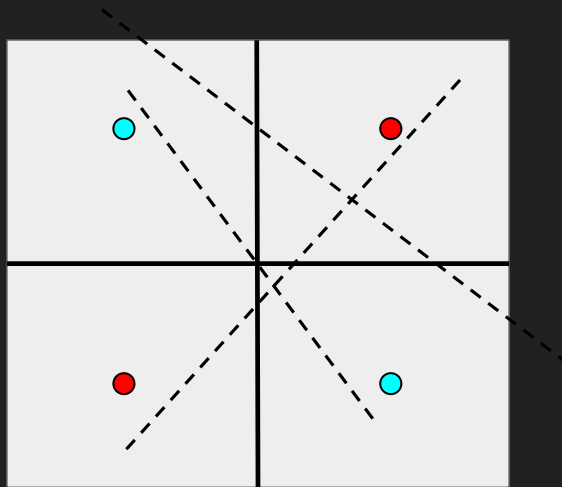


Papert

Picture credits: [Wikipedia](#), [Wikipedia](#)

Limits of a single hyperplane: XOR

- 2-d points labeled with the XOR rule cannot be learned by a Perceptron!
 - **Recall:** the XOR (exclusive or) of (x_1, x_2) is +1 when exactly one input is activated
- Many other simple deterministic labeling rules pose similar issues, e.g., parity



Crisis: An AI Winter

- How is a Perceptron supposed to “walk, talk, see...” if it cannot even learn XOR?
- These types of questions put a damper on the first AI hype cycle



Picture credits: https://www.reddit.com/r/singularity/comments/14cz0uf/how_likely_is_an_ai_winter/

What to do?

- There was consensus early on that the Perceptron was **too simple as an architecture**

*“More sophisticated approaches must therefore be undertaken. A number of alternatives are possible . . . The most attractive appears to be multiple-layer logical circuit arrangements, since it is known that **any function can thereby be realized**. . .” — J. K. Hawkins, 1961*

- At Stanford, Bernard Widrow organized a group to investigate **multilayer** networks

Quote taken from: MIKEL OLAZARAN, “A Sociological History of the Neural Network Controversy”, 1993

Visions of Multilayer Fully Connected Neural Networks

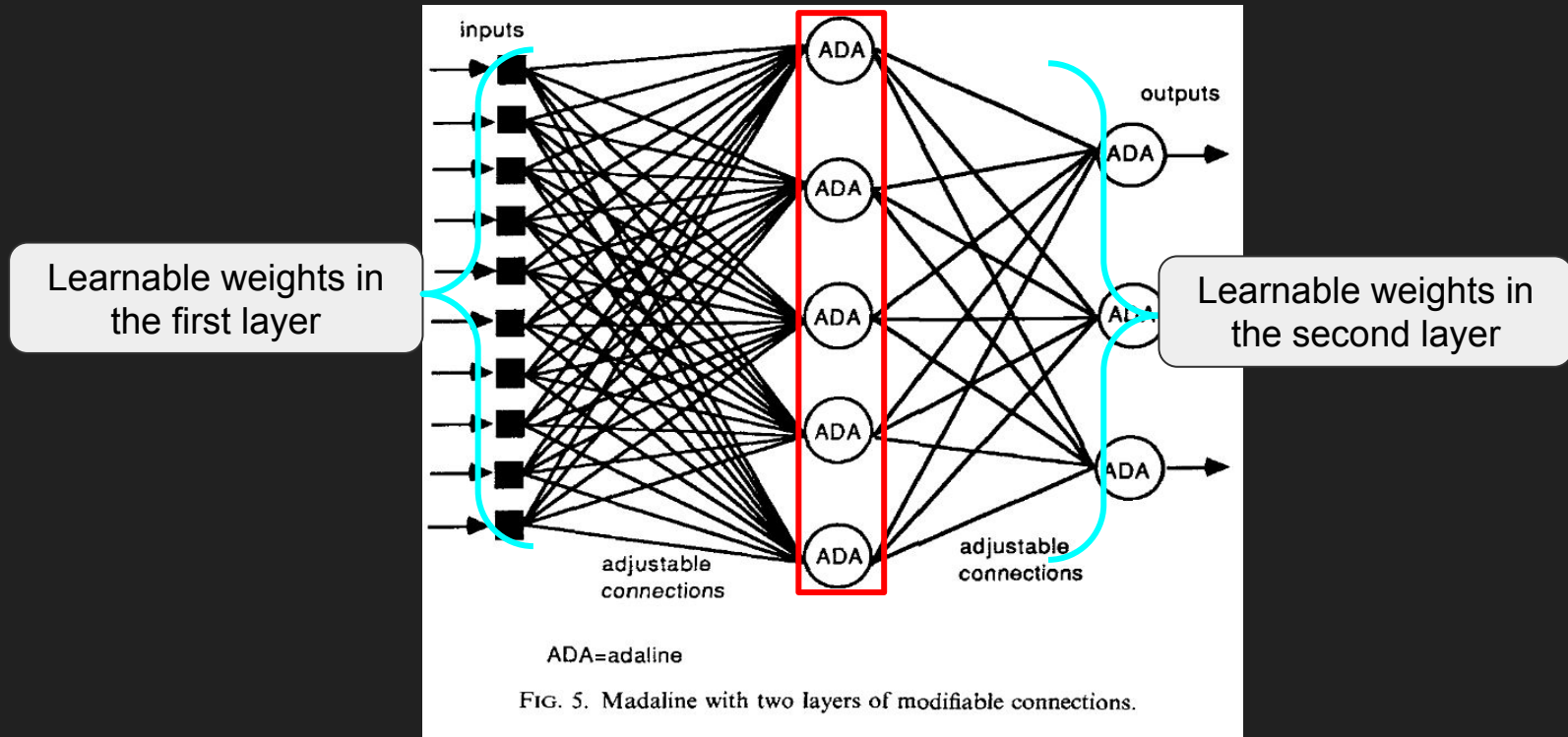
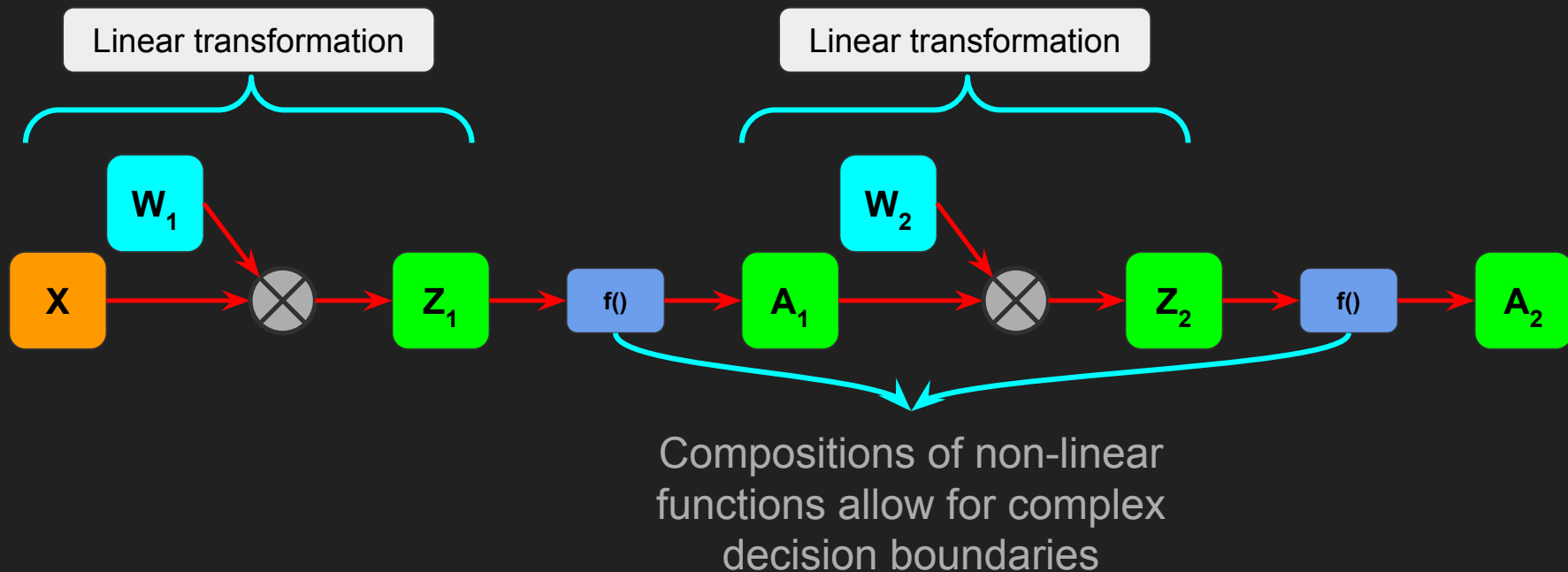


Figure credit: Widrow, 1962

More layers, more expressivity

- With more layers comes more opportunities for *non-linear* activations



How to train Multilayer Perceptrons?

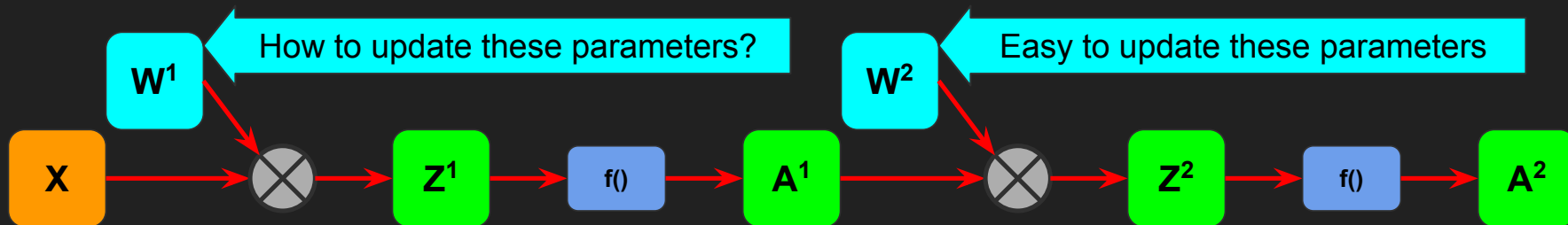
- Multilayer networks were appealing as they offered provable universal function approximation abilities
- However, the revolutionary update rule used to train the perceptron architecture does not extend to multilayer networks

We tried to adapt layered [i.e., multilayer] neural nets, but we never succeeded. We were able to adapt a two-layer network, the madaline, where the first layer was adaptive but the second layer was fixed. By knowing the nature of the second layer we were able to make rules for adapting the first layer. But if the second layer was completely free to do what it wanted, we didn't have any general rule for adapting the first layer . . . (Widrow, interview)

Quote taken from: MIKEL OLAZARAN, "A Sociological History of the Neural Network Controversy", 1993

The 1st Quest for Depth

- Many researchers, including Widrow and his group, sought an algorithm for training multilayer networks
- However, such an algorithm would not emerge until the mid 1980s
- This prompted many researchers to pursue rule-based symbolic approaches to artificial intelligence during the AI winter



Backpropagation

End of the 1st Quest for Depth

- The **backpropagation algorithm** was introduced by Rumelhart et. al., in 1986
 - Origins of the idea can be traced back to the 1940s in control theory
 - Paul Werbos is credited with developing the precursors to backprop in the 1970s
- Allowed multilayer (or deep) neural networks to be training using gradient descent and other analytical optimization algorithms
- Revitalized deep learning research

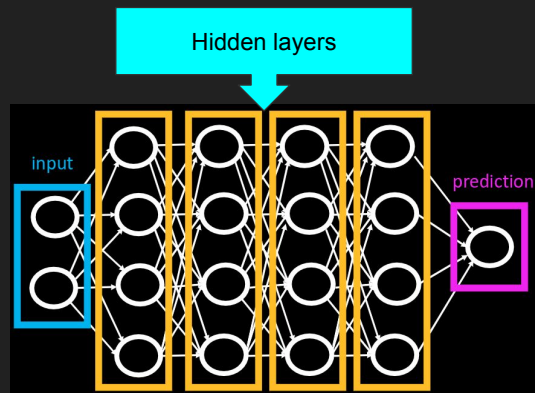


Rumelhart in 1991

Picture credit: [Wikipedia](#)

Some Notation

- A multilayer (fully connected) neural network is a sequence of *layers*, vectors of neurons
- $A^{[i]}$ will denote *activations*
 - We will use $Z^{[i]}$ to refer to the intermediate *features*



Picture credits: <https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/03-Fully-Connected-NN.pdf>

How to train your Multilayer Neural Network

- Three key steps:
 - Forward pass — compute prediction based on input as a composition of differentiable functions
 - Compute a *loss function* — real-valued function of two inputs describing the correctness of the prediction
 - Backward pass — update the parameters of the model using the chain rule for derivatives
- Critically, we need the loss function to be differentiable
- **Recall:** the chain rule allows us to compute the derivative of a composition of functions using the derivatives of the original functions
- By computing gradients of the loss w.r.t each parameter, we can apply ***gradient descent*** to adjust the model

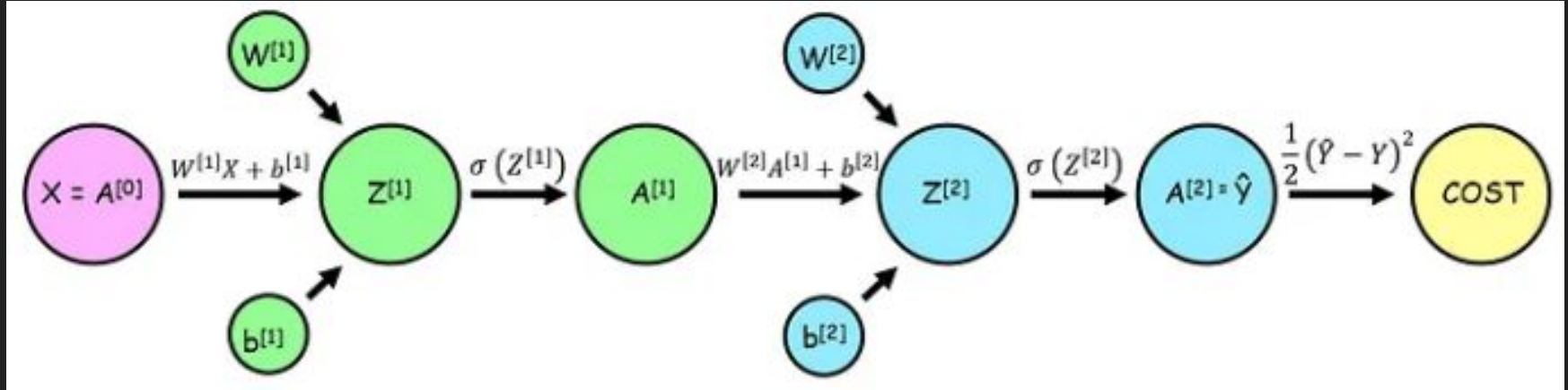
Chain Rule: Review

- **Recall:** if we're given two differentiable functions $f(x)$, $g(y)$, then:

$$[g(f(x))]' = g'(f(x)) * f'(x)$$

- Important point: there's a *multiplication* involved!

Forward Pass



We sequentially compute the A values using the equation:

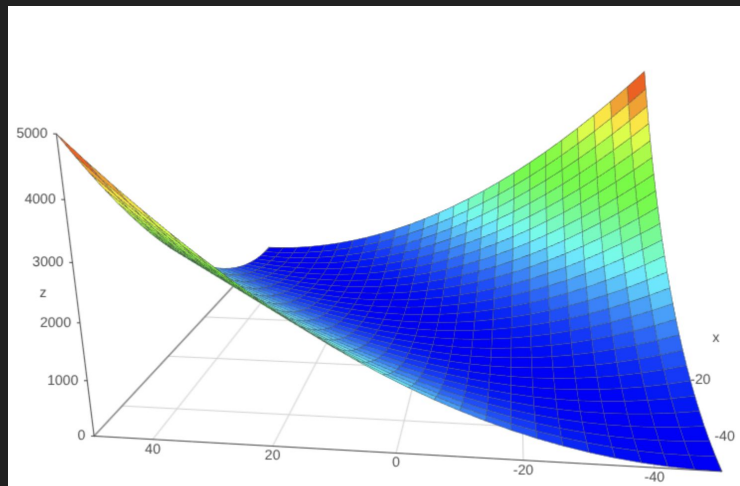
$$A^i = \sigma(W^i X^{i-1} + B^i)$$

Loss Function

- For an input sample (X, Y) , and model prediction $\hat{Y}$

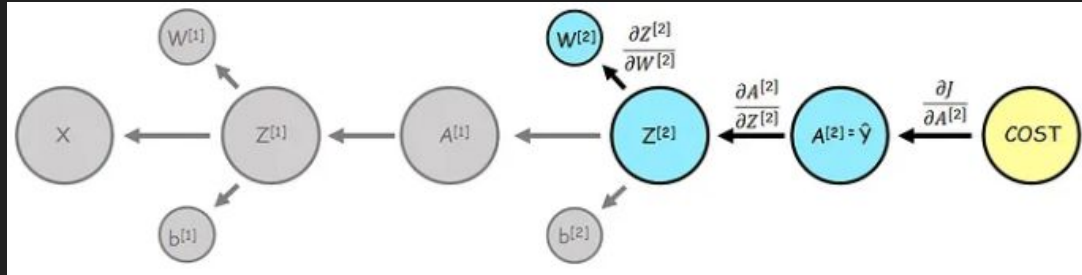
$$J = L(\hat{Y}, Y) = \frac{1}{2}(\hat{Y} - Y)^2$$

- This is known as *mean squared error loss*
- It's “nice” — differentiable in both variables



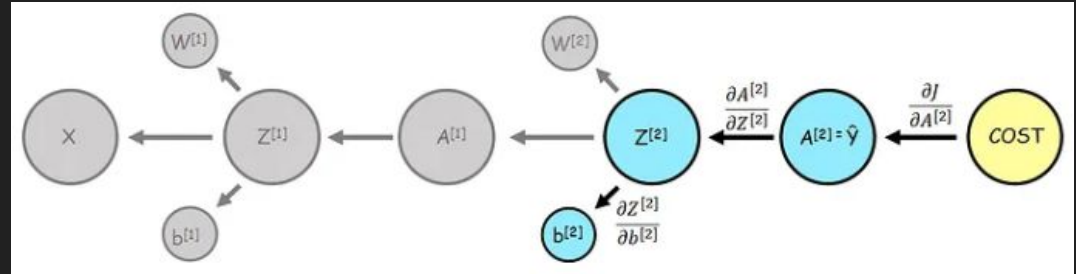
Plot created in: <https://academo.org/demos/3d-surface-plotter/>

Backward Pass: Step 1



$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

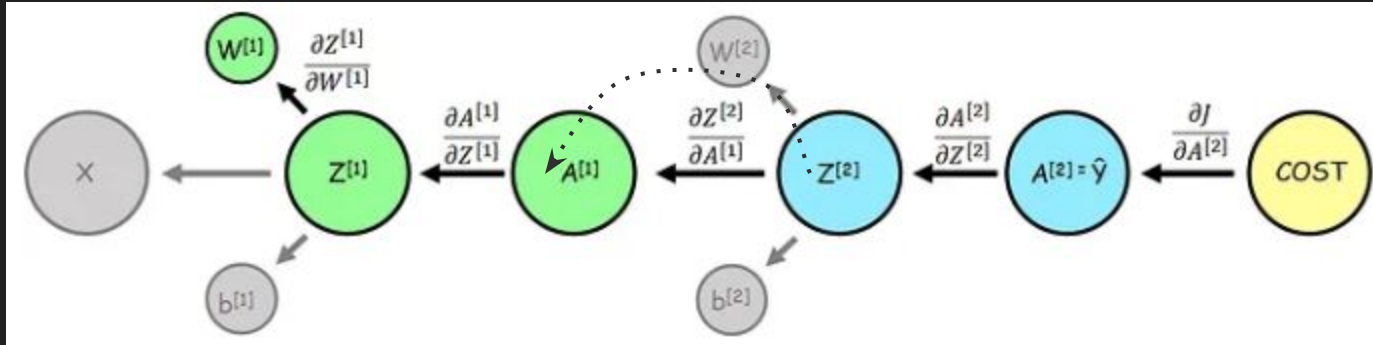
$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$



Picture credits: <https://medium.com/data-science/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce>

Backward Pass: Step 2

- Key idea: use chain to “jump back” to previous layer



$$\frac{\partial J}{\partial W^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{\text{Previously calculated}} \frac{\partial Z^{[1]}}{\partial W^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot X$$

Backward Pass: Step 3

- Finally, we update the parameters of the model using a *learning rate*, α

$$\begin{aligned} W_{new}^{[1]} &= W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}} & | & & b_{new}^{[1]} &= b_{old}^{[1]} - \alpha \frac{dJ}{db^{[1]}} \\ W_{new}^{[2]} &= W_{old}^{[2]} - \alpha \frac{dJ}{dW^{[2]}} & | & & b_{new}^{[2]} &= b_{old}^{[2]} - \alpha \frac{dJ}{db^{[2]}} \end{aligned}$$

- Learning rate controls the “jumpy-ness” of the iterative parameter changes
 - High learning rate: large parameter updates, faster learning or divergence
 - Low learning rate: small parameter updates, slower learning or improved convergence

Picture credits: <https://medium.com/data-science/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce>

Gradient Descent: Intuition



- We want to find **good minima** in the **loss landscape**
- Hope that by iteratively taking *steps* (of size α), we can find such values

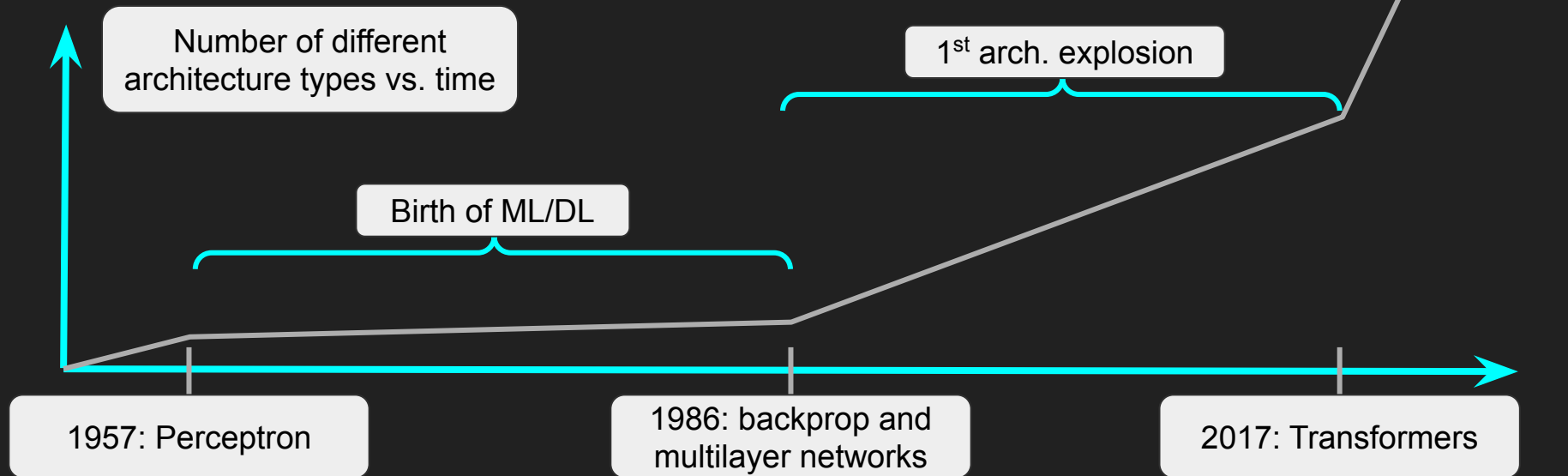
Picture credits: <https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/03-Fully-Connected-NN.pdf>

Tips for training your Deep Neural Network

- As models become deeper and their decision boundaries become less linear, their loss landscapes become complicated and non-convex
- Key points:
 - Need to choose (and potentially adjust during training) the learning rate carefully
 - Need to select “good” input/output samples
 - Need the model to have enough layers and neurons
- Intuition:
 - When hiking a mountain, big steps good. When descending stairs, small steps good.

The First Architecture Explosion (1986-2017)

- Backpropagation prompted vastly renewed interest in DNN research
- Inspired researchers to create many new types of neural networks



Limits of Backprop

- Just as how the Perceptron is a “simple approximation” of an artificial neuron, backprop is a “simple approximation” of the process of learning
 - In particular, the *credit assignment problem* — deciding which params. to adjust

The learning procedure, in its current form, is not a plausible model of learning in brains. However, applying the procedure to various tasks shows that interesting internal representations can be constructed by gradient descent in weight-space, and this suggests that it is worth looking for more biologically plausible ways of doing gradient descent in neural networks.

- It took another ~30 years (2022) for these ideas to develop into alternative parameter update algorithms, such as [Forward-Forward](#) and [PEPITA](#)

Quote from: Rumelhart et. al., “Learning representations by back-propagating errors”, Nature, 1986

Summary

- People have been fascinated by the idea of intelligent machines for a long time
- The perceptron architecture (~1960) developed a way to teach machines using data samples
- Limits of the perceptron inspired research into *deep* neural networks
- Such networks could not be trained until the 1980s
- The backpropagation algorithm revolutionized deep learning and created the first architectural explosion

Coming up...

- Many architectures from the first explosion
 - Convolutional neural networks
 - Recurrent neural networks
 - Residual networks
- Attention-based networks
 - Transformers
 - The second architecture explosion
- Recent theories of deep neural networks
 - Geometry, topology, and combinatorics, oh my!